SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO4: *Examine intermediate code generation techniques to enhance code optimization (BL4)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks:50**

Annexure A

INDUSTRY PROBLEM- SOLVING TASK

Code of Conduct:

I ANIKSHA EVANJALIN M(192521308) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ANIKSHA

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	**/ 25**	**/ 20**	**/ 20**	**/ 10**	**/ 100**

Annexure A

INDUSTRY PROBLEM- SOLVING TASK

1. Banking Transaction Processing System
A banking software company is developing a compiler for a domain-specific scripting language used in transaction processing systems. The compiler must translate variable declarations, assignment statements, and Boolean expressions efficiently to avoid transaction failures during peak hours. The system also uses case statements to classify transactions such as deposit, withdrawal, and loan processing. During testing, incorrect intermediate code generation caused delays in transaction execution.
Tasks:
(i) Design suitable intermediate code for the given assignment and Boolean expressions. *(K3)*
(ii) Explain how backpatching can help in handling conditional transaction validation. *(K4)*
(iii) Analyze the role of intermediate languages in improving banking software portability. *(K5)*

---

2. Smart Traffic Control System
An automotive company is building a smart traffic management application where signals are controlled automatically based on traffic density. The compiler used in the embedded controller must process Boolean expressions and case statements for different traffic conditions. Due to inefficient intermediate code generation, the response time of traffic signals increased significantly during rush hours. The development team plans to optimize procedure calls and assignment statements in the compiler design.
Tasks:
(i) Construct intermediate code for the traffic signal control logic. *(K3)*
(ii) Explain how procedure calls improve modularity in the traffic control application. *(K4)*
(iii) Evaluate the importance of compiler optimization in real-time embedded systems. *(K5)*

---

3. E-Commerce Product Recommendation Engine

An e-commerce company uses a rule-based recommendation engine developed using a custom programming language. The compiler processes declarations, assignment statements, and Boolean conditions to generate recommendations dynamically. During seasonal sales, delays occurred because of improper handling of case statements and inefficient intermediate code generation. The organization wants to redesign the compiler to improve execution speed and maintainability.
Tasks:
(i) Generate intermediate representation for the recommendation logic. *(K3)*
(ii) Discuss how case statements can efficiently handle product category selection. *(K4)*
(iii) Analyze the role of compiler design in improving large-scale e-commerce applications. *(K5)*

---

4. Hospital Patient Monitoring System
A healthcare technology company is developing a patient monitoring system that continuously checks vital signs using embedded software. The compiler used in the system must efficiently evaluate Boolean expressions and assignment statements for emergency alerts. Improper backpatching in conditional branching caused delayed alarm generation during

critical patient conditions. The software team is redesigning the compiler for faster and more reliable intermediate code execution.
Tasks:
(i) Design intermediate code for patient condition monitoring logic. *(K3)*
(ii) Explain the significance of backpatching in emergency alert systems. *(K4)*
(iii) Evaluate how compiler optimization improves reliability in healthcare applications. *(K5)*

---

5. Online Airline Reservation System
An airline company is modernizing its reservation platform using a custom scripting language for ticket booking and seat allocation. The compiler must handle declarations, assignment statements, and procedure calls efficiently to manage thousands of booking requests simultaneously. During testing, errors in intermediate code generation affected seat allocation and payment confirmation processes. The software architects aim to improve compiler performance for large-scale distributed applications.
Tasks:
(i) Develop intermediate code for ticket booking and seat allocation operations. *(K3)*
(ii) Explain how procedure calls support modular reservation management. *(K4)*
(iii) Analyze the role of intermediate languages in distributed airline reservation systems. *(K5)*

Set 2

1. Industrial Robot Automation System

A manufacturing company is developing an industrial robot control system where robotic arms perform assembly operations automatically. The compiler used in the embedded software processes declarations, assignment statements, and Boolean expressions to coordinate robotic movement and safety checks. During production, delays occurred due to inefficient handling of procedure calls and intermediate code generation. The company plans to redesign the compiler to improve execution speed and synchronization between robotic units.

Tasks:

(i) Design intermediate code for robotic movement and safety validation logic. *(K3)*
(ii) Explain how procedure calls improve modularity in industrial automation systems. *(K4)*
(iii) Analyze the role of compiler optimization in improving manufacturing efficiency. *(K5)*

---

2. Cybersecurity Intrusion Detection System

A cybersecurity firm is developing an intrusion detection system that continuously monitors network traffic for suspicious activities. The compiler processes Boolean expressions, assignment statements, and case statements to identify different categories of cyberattacks. During real-time monitoring, inefficient intermediate code generation increased system response time and delayed threat detection. The development team intends to enhance compiler efficiency for faster and more reliable security analysis.

Tasks:

(i) Generate intermediate representation for network threat detection logic. *(K3)*
(ii) Discuss how case statements can efficiently classify multiple cyberattack scenarios. *(K4)*
(iii) Evaluate the importance of compiler optimization in real-time cybersecurity applications. *(K5)*

---

3. Digital Payment Gateway System

A fintech company is building a digital payment gateway to process online transactions securely and efficiently. The compiler used in the transaction engine must evaluate Boolean conditions, assignment statements, and procedure calls during payment verification and fraud detection. During peak transaction periods, improper backpatching caused delays in payment confirmation and rollback operations. The organization plans to redesign the compiler to improve transaction reliability and scalability.

Tasks:

(i) Construct intermediate code for payment verification and fraud detection logic. *(K3)*
(ii) Explain how backpatching supports conditional transaction processing. *(K4)*
(iii) Analyze the role of compiler design in ensuring secure and scalable fintech applications. *(K5)*

---

4. Smart Agriculture Monitoring System

An agritech company is developing a smart agriculture platform that monitors soil moisture, temperature, and irrigation systems automatically. The compiler in the embedded controller processes declarations, assignment statements, and Boolean expressions to activate irrigation based on environmental conditions. Inefficient handling of intermediate code caused delays in irrigation control during critical farming conditions. The software team aims to improve compiler performance for real-time agricultural automation.

Tasks:

(i) Develop intermediate code for irrigation control and environmental monitoring. *(K3)*
(ii) Explain the role of Boolean expressions in automated farming systems. *(K4)*
(iii) Evaluate how compiler optimization enhances efficiency in smart agriculture applications. *(K5)*

---

5. Cloud-Based Video Streaming Platform

A multimedia company is developing a cloud-based video streaming platform that dynamically adjusts video quality based on network conditions. The compiler used in the streaming engine processes case statements, assignment statements, and procedure calls for resource allocation and bandwidth management. During high user traffic, inefficient intermediate code generation caused buffering delays and reduced streaming quality. The company plans to redesign the compiler to improve scalability and performance.

Tasks:

(i) Design intermediate code for bandwidth allocation and quality adaptation logic. *(K3)*
(ii) Explain how case statements help manage multiple streaming quality levels. *(K4)*
(iii) Analyze the significance of compiler optimization in cloud-based multimedia systems. *(K5)*

Set 3

1. Autonomous Drone Delivery System
A logistics company is developing an autonomous drone delivery system to transport packages across urban areas. The compiler embedded in the drone controller processes declarations, assignment statements, and Boolean expressions to manage navigation, obstacle detection, and battery monitoring. During testing, inefficient intermediate code generation caused delays in route adjustment and emergency landing procedures. The organization plans to optimize the compiler for reliable real-time drone operations.
Tasks:
(i) Design intermediate code for drone navigation and obstacle detection logic. *(K3)*
(ii) Explain how Boolean expressions support autonomous decision-making in drones. *(K4)*
(iii) Analyze the importance of compiler optimization in real-time delivery systems. *(K5)*

---

2. Railway Reservation and Scheduling System

A railway management company is modernizing its reservation and scheduling platform using a custom programming language. The compiler processes declarations, assignment statements, case statements, and procedure calls for seat allocation, train scheduling, and passenger status updates. During peak booking hours, improper intermediate code generation caused delays in reservation confirmation and ticket cancellation. The development team aims to redesign the compiler to improve scalability and transaction efficiency.

Tasks:

(i) Construct intermediate code for reservation and scheduling operations. *(K3)*

(ii) Discuss how case statements efficiently handle passenger ticket categories. *(K4)*

(iii) Evaluate the role of compiler optimization in large-scale transportation systems. *(K5)*

---

3. Intelligent Energy Management System

An energy company is building a smart grid monitoring system that automatically controls electricity distribution based on power consumption. The compiler in the embedded controller evaluates Boolean expressions and assignment statements to monitor load balancing and fault detection. During high-demand periods, inefficient backpatching delayed fault recovery and power redistribution. The company plans to enhance compiler performance to improve reliability and energy efficiency.

Tasks:

(i) Generate intermediate code for load balancing and fault detection logic. *(K3)*

(ii) Explain the significance of backpatching in smart energy management systems. *(K4)*

(iii) Analyze how compiler optimization improves the performance of smart grids. *(K5)*

---

4. AI-Based Fraud Detection System

A financial institution is developing an AI-based fraud detection platform that continuously analyzes transaction patterns. The compiler processes declarations, assignment statements, and Boolean conditions to identify suspicious activities and trigger alerts. During large-scale transaction processing, inefficient procedure calls increased response time and delayed fraud notifications. The software architects plan to redesign the compiler for faster analysis and improved scalability.

Tasks:

(i) Develop intermediate code for fraud detection and alert generation logic. *(K3)*

(ii) Explain how procedure calls improve modularity in fraud detection systems. *(K4)*

(iii) Evaluate the role of compiler design in secure financial applications. *(K5)*

---

5. Smart Warehouse Inventory Management System

A retail company is implementing a smart warehouse inventory system that automatically tracks product movement and stock levels. The compiler processes assignment statements, Boolean expressions, and case statements to update inventory records and manage automated storage systems. During seasonal sales, delays in intermediate code execution affected stock synchronization and order fulfillment. The company plans to optimize the compiler to improve inventory accuracy and warehouse efficiency.

Tasks:

(i) Design intermediate code for inventory tracking and stock update operations. *(K3)*

(ii) Explain how case statements help manage multiple product categories efficiently. *(K4)*

(iii) Analyze the importance of compiler optimization in warehouse automation systems. *(K5)*

Set 4

1. Smart Home Automation System

A home automation company is developing an intelligent control system that automatically manages lighting, temperature, and security devices. The compiler embedded in the controller processes declarations, assignment statements, and Boolean expressions to activate devices based on sensor inputs and user preferences. During real-time execution, inefficient intermediate code generation caused delays in device response and security alerts. The development team plans to redesign the compiler to improve automation efficiency and reliability.

Tasks:

(i) Design intermediate code for smart device control and sensor monitoring logic. *(K3)*

(ii) Explain how Boolean expressions support automated decision-making in smart homes. *(K4)*

(iii) Analyze the role of compiler optimization in improving home automation systems. *(K5)*

---

2. Online Examination Management System

An educational technology company is building an online examination platform that evaluates students automatically and manages exam scheduling. The compiler processes declarations, assignment statements, case statements, and procedure calls for student authentication and result generation. During large-scale online examinations, inefficient intermediate code execution caused delays in question loading and score processing. The organization plans to optimize the compiler to improve scalability and performance.

Tasks:

(i) Construct intermediate code for student authentication and evaluation logic. *(K3)*

(ii) Discuss how case statements help manage different examination categories. *(K4)*

(iii) Evaluate the importance of compiler optimization in large-scale online examination systems. *(K5)*

---

3. Intelligent Weather Forecasting System

A meteorological department is developing an intelligent weather forecasting application that analyzes environmental data in real time. The compiler processes assignment statements and Boolean expressions to monitor temperature, humidity, and rainfall conditions. During severe weather conditions, improper backpatching delayed alert generation and emergency notifications. The software engineers aim to redesign the compiler to improve prediction accuracy and system responsiveness.

Tasks:

(i) Generate intermediate code for weather monitoring and alert generation logic. *(K3)*

(ii) Explain the role of backpatching in handling emergency weather alerts. *(K4)*

(iii) Analyze how compiler optimization enhances real-time forecasting applications. *(K5)*

---

4. Automated Library Management System

A university is implementing an automated library management system to handle book issuance, returns, and digital catalog management. The compiler processes declarations, assignment statements, and procedure calls for member verification and book tracking operations. During semester examinations, inefficient intermediate code generation delayed transaction updates and fine calculations. The university plans to improve compiler efficiency for faster and more reliable library services.

Tasks:

(i) Develop intermediate code for book issue and return operations. *(K3)*

(ii) Explain how procedure calls improve modularity in library management systems. *(K4)*

(iii) Evaluate the significance of compiler optimization in educational information systems. *(K5)*

---

5. Industrial Water Quality Monitoring System

An environmental engineering company is developing a water quality monitoring system that continuously checks pH levels, contamination, and chemical composition in industrial plants. The compiler in the embedded controller evaluates Boolean expressions, assignment statements, and case statements for automated monitoring and alert generation. During critical contamination events, inefficient intermediate code execution delayed warning notifications and corrective actions. The company aims to redesign the compiler to improve reliability and response time.

Tasks:

(i) Design intermediate code for contamination detection and alert handling logic. *(K3)*

(ii) Explain how case statements efficiently manage different water quality conditions. *(K4)*

(iii) Analyze the importance of compiler optimization in industrial environmental monitoring systems. *(K5)*

Set 5

1. Smart Parking Management System

A smart city development company is designing an automated parking management system to monitor vehicle entry, slot allocation, and parking fee calculation. The compiler embedded in the control software processes declarations, assignment statements, Boolean expressions, and case statements to manage parking availability and security verification. During peak hours, inefficient intermediate code generation caused delays in slot assignment and payment confirmation. The company plans to redesign the compiler to improve real-time response and system efficiency.

Tasks:

(i) Design intermediate code for parking slot allocation and fee calculation logic. *(K3)*

(ii) Explain how Boolean expressions and case statements help manage parking operations efficiently. *(K4)*

(iii) Analyze the role of compiler optimization in improving smart parking management systems. *(K5)*

---

2. Digital Healthcare Appointment System

A healthcare organization is developing a digital appointment management platform to schedule doctor consultations and manage patient records. The compiler processes declarations, assignment statements, and procedure calls to handle appointment booking, cancellation, and patient verification. During high patient traffic, inefficient intermediate code execution caused delays in appointment confirmation and report generation. The development team plans to optimize the compiler for faster and more reliable healthcare services.

Tasks:

(i) Construct intermediate code for appointment scheduling and patient verification logic. *(K3)*

(ii) Explain how procedure calls improve modularity in healthcare management systems. *(K4)*

(iii) Evaluate the importance of compiler optimization in digital healthcare applications. *(K5)*

---

3. Automated Toll Collection System

A transportation authority is implementing an automated toll collection system that identifies vehicles and processes payments electronically. The compiler embedded in the toll

management software evaluates Boolean expressions and assignment statements for vehicle classification and transaction validation. During festival seasons, improper backpatching caused delays in payment verification and traffic management. The organization aims to redesign the compiler to improve transaction speed and operational efficiency.

Tasks:

(i) Generate intermediate code for vehicle identification and toll payment processing. *(K3)*

(ii) Explain the significance of backpatching in automated toll management systems. *(K4)*

(iii) Analyze how compiler optimization improves large-scale transportation applications. *(K5)*

---

4. Intelligent Fire Detection and Alarm System

A safety engineering company is developing an intelligent fire detection system that continuously monitors smoke and temperature sensors in industrial buildings. The compiler processes Boolean expressions, assignment statements, and case statements to activate alarms and emergency protocols. During emergency situations, inefficient intermediate code execution delayed warning notifications and evacuation procedures. The software engineers plan to optimize the compiler for faster real-time response.

Tasks:

(i) Develop intermediate code for fire detection and emergency alert logic. *(K3)*

(ii) Explain how case statements efficiently handle multiple emergency conditions. *(K4)*

(iii) Evaluate the role of compiler optimization in industrial safety systems. *(K5)*

---

5. AI-Based Customer Support Chatbot

A software company is building an AI-based customer support chatbot to handle user queries and provide automated responses. The compiler processes declarations, assignment statements, Boolean expressions, and procedure calls to analyze user requests and generate responses dynamically. During large-scale customer interactions, inefficient intermediate code generation caused delays in response delivery and reduced system performance. The organization plans to redesign the compiler to improve scalability and user experience.

Tasks:

(i) Design intermediate code for chatbot query processing and response generation. *(K3)*

(ii) Explain how procedure calls improve modularity in AI-based chatbot systems. *(K4)*

(iii) Analyze the importance of compiler optimization in intelligent customer support applications. *(K5)*

ANSWERS

SET 1

1. Banking Transaction Processing System

(i) Intermediate code for assignment and Boolean expressions:

Suppose the transaction logic is:

balance = balance - amount
valid = (amount > 0) AND (balance >= amount)

Three-address code:

T1 = balance - amount
balance = T1

T2 = amount > 0
T3 = balance >= amount
T4 = T2 AND T3
valid = T4

For a case statement:

```
case type
  deposit: balance = balance + amount
  withdraw: balance = balance - amount
  loan: processLoan()
```

Intermediate code:

```
if type == deposit goto L1
if type == withdraw goto L2
if type == loan goto L3
goto L4

L1: T1 = balance + amount
    balance = T1
    goto L4

L2: T2 = balance - amount
    balance = T2
    goto L4

L3: call processLoan
    goto L4
```

L4: ...

(ii) Backpatching for conditional transaction validation:

Backpatching is used when the compiler does not yet know the target address of a conditional jump.

For example:

```
if amount > 0 AND balance >= amount
   approve transaction
else
   reject transaction
```

Intermediate code:

```
if amount > 0 goto _
goto _
if balance >= amount goto _
goto _
```

The blank targets are filled later using backpatching.

Thus:

- true list is patched to the approval code.
- false list is patched to the rejection code.
- It helps generate correct conditional branches.
- It prevents incorrect jumps that could cause transaction failures.

(iii) Role of intermediate languages in banking software portability:

Intermediate languages such as Three-Address Code, bytecode, or an intermediate representation (IR) provide a platform-independent form of the program.

Advantages:

1. The same banking program can run on different hardware platforms.
2. The compiler can optimize the IR before generating machine code.
3. Changes in hardware do not require redesigning the entire source language.
4. It improves maintainability of banking software.
5. It supports different operating systems and processor architectures.
6. Optimized IR can improve transaction processing speed.

2. Smart Traffic Control System

(i) Intermediate code for traffic signal control:

Suppose:

```
if traffic_density > 80
   signal = RED
else if traffic_density > 50
   signal = YELLOW
else
   signal = GREEN
```

Three-address code:

```
if traffic_density > 80 goto L1
goto L2

L1: signal = RED
   goto L4

L2: if traffic_density > 50 goto L3
   goto L5

L3: signal = YELLOW
   goto L4

L5: signal = GREEN

L4: ...
```

This IR allows the embedded controller to quickly determine the appropriate signal.

(ii) Procedure calls and modularity:

Procedure calls allow different traffic-control functions to be separated into modules.

Example:

```
call readTrafficSensor
call calculateDensity
```

```
call controlSignal
call emergencyOverride
```

Benefits:

1. Each function performs one specific task.
2. Code becomes easier to understand.
3. Individual procedures can be tested separately.
4. Code can be reused.
5. Maintenance becomes easier.
6. Emergency traffic handling can be added without changing the entire system.

(iii) Importance of compiler optimization in real-time embedded systems:

Compiler optimization is very important because traffic signals must respond within a short time.

Optimization:

- Reduces execution time.
- Reduces unnecessary instructions.
- Reduces memory usage.
- Improves processor efficiency.
- Reduces power consumption.
- Makes signal response faster.
- Helps meet real-time deadlines.

Therefore, optimized intermediate code is essential for reliable smart traffic control.

3. E-Commerce Product Recommendation Engine

(i) Intermediate representation for recommendation logic:

Suppose:

```
if category == "Electronics" AND price < 50000
   recommendation = "Laptop"
else if category == "Clothing"
   recommendation = "Fashion Products"
else
   recommendation = "General Products"
```

Three-address code:

```
T1 = category == "Electronics"
T2 = price < 50000
```

```
T3 = T1 AND T2

if T3 goto L1
goto L2

L1: recommendation = "Laptop"
    goto L4

L2: if category == "Clothing" goto L3
    goto L5

L3: recommendation = "Fashion Products"
    goto L4

L5: recommendation = "General Products"

L4: ...
```

(ii) Case statements for product category selection:

Case statements are useful when a recommendation depends on different product categories.

Example:

```
case category
    Electronics: recommend electronics
    Clothing: recommend clothes
    Books: recommend books
    Grocery: recommend groceries
```

They provide:

1. Clear category-based selection.
2. Easy addition of new categories.
3. Better readability.
4. Efficient branching.
5. Easier maintenance.
6. Faster execution when implemented using optimized jump tables.

(iii) Role of compiler design in large-scale e-commerce applications:

Compiler design improves e-commerce applications by:

- Generating efficient intermediate code.
- Optimizing Boolean expressions and assignments.
- Reducing execution time.
- Improving memory utilization.
- Supporting scalable recommendation systems.
- Making the custom language easier to maintain.

- Improving response time during heavy sales traffic.

Thus, good compiler design helps an e-commerce system handle a large number of users efficiently.

4. Hospital Patient Monitoring System

(i) Intermediate code for patient monitoring:

Suppose:

```
if heart_rate > 120 OR oxygen_level < 90
   alarm = ON
else
   alarm = OFF
```

Three-address code:

```
T1 = heart_rate > 120
if T1 goto L1

T2 = oxygen_level < 90
if T2 goto L1

goto L2

L1: alarm = ON
   call emergencyAlert
   goto L3

L2: alarm = OFF

L3: ...
```

This code allows emergency conditions to be checked quickly.

(ii) Significance of backpatching in emergency alert systems:

Backpatching is important because conditional jumps may not have their final target addresses during the first stage of code generation.

For example:

```
if heart_rate > 120 goto _
goto _
```

Later, the compiler patches the first jump to the emergency alarm code.

Benefits:

1. Produces correct conditional branching.
2. Helps connect emergency conditions to alarm procedures.
3. Prevents incorrect jump destinations.
4. Supports nested Boolean conditions.
5. Reduces errors in intermediate code.
6. Helps generate reliable alarm execution.

In healthcare systems, correct branching is especially important because a wrong jump could delay an emergency alert.

(iii) Compiler optimization and healthcare reliability:

Compiler optimization can improve healthcare applications by:

- Reducing response time.
- Making emergency alerts faster.
- Removing unnecessary calculations.
- Reducing memory requirements.
- Improving processor utilization.
- Producing predictable execution.
- Helping embedded devices operate efficiently.

Therefore, optimized compiler-generated code improves both performance and reliability of patient monitoring systems.

5. Online Airline Reservation System

(i) Intermediate code for ticket booking and seat allocation:

Suppose:

```
if seat_available == true
   seat = allocateSeat()
   ticket = bookTicket()
   payment = processPayment()
else
   status = "No Seat Available"
```

Three-address code:

```
if seat_available == true goto L1
goto L5
```

```
L1: call allocateSeat
    seat = RET

    call bookTicket
    ticket = RET

    call processPayment
    payment = RET

    status = "Booking Confirmed"
    goto L6

L5: status = "No Seat Available"

L6: ...
```

The intermediate code represents the booking operations before final machine code generation.

(ii) Procedure calls for modular reservation management:

Procedure calls divide the reservation system into independent functions.

For example:

```
call checkSeatAvailability
call allocateSeat
call bookTicket
call processPayment
call sendConfirmation
```

Advantages:

1. Each operation is handled separately.
2. Functions can be reused.
3. The system is easier to test.
4. Maintenance becomes simpler.
5. Payment and booking modules can be independently updated.
6. Errors can be isolated to individual modules.
7. Large reservation systems become easier to manage.

(iii) Role of intermediate languages in distributed airline reservation systems:

Intermediate languages provide a common representation between the source program and different target machines.

They help airline systems by:

1. Supporting different server architectures.
2. Improving application portability.
3. Allowing compiler optimizations before machine-code generation.
4. Reducing development effort for multiple platforms.
5. Supporting distributed servers and cloud environments.
6. Improving execution speed.
7. Making large reservation applications easier to maintain.

SET 2

1. Industrial Robot Automation System

(i) Intermediate code for robotic movement and safety validation:

Suppose the robot moves only when the safety condition is satisfied.

```
T1 = arm_position == target_position
T2 = safety_sensor == ON
T3 = emergency_stop == OFF
T4 = T2 AND T3
T5 = T1 AND T4

if T5 goto L1
goto L2

L1: call moveRobot
    status = "Movement Completed"
    goto L3

L2: status = "Movement Stopped"

L3: ...
```

This intermediate code first checks the robot position and safety conditions. If all conditions are true, the robot movement procedure is called.

(ii) Procedure calls and modularity:

Procedure calls divide the robot-control system into smaller modules.

Examples:

```
call checkSafety
call moveRobot
call pickObject
```

```
call placeObject
call emergencyStop
```

Advantages:

- Each procedure performs a specific task.
- The same procedure can be reused.
- Testing becomes easier.
- Errors can be isolated.
- Maintenance becomes simpler.
- Different robotic units can use common procedures.
- The overall program becomes more organized.

(iii) Role of compiler optimization in manufacturing efficiency:

Compiler optimization improves industrial automation by:

- Reducing execution time.
- Producing faster machine code.
- Reducing unnecessary instructions.
- Improving synchronization between robots.
- Reducing memory usage.
- Improving processor utilization.
- Providing faster response to safety conditions.

Therefore, compiler optimization helps increase production speed, accuracy, safety, and overall manufacturing efficiency.

2. Cybersecurity Intrusion Detection System

(i) Intermediate representation for network threat detection:

Suppose the system checks suspicious traffic:

```
T1 = packet_rate > 1000
T2 = failed_login > 5
T3 = unusual_port == TRUE
T4 = T1 OR T2
T5 = T4 OR T3

if T5 goto L1
goto L2

L1: alert = "Threat Detected"
```

```
    call blockTraffic
    goto L3

L2: alert = "Normal Traffic"

L3: ...
```

This intermediate representation allows the compiler to efficiently process network conditions and generate appropriate actions.

(ii) Case statements for cyberattack classification:

Case statements can classify different types of attacks based on an attack code.

```
case attack_type

1: alert = "DDoS Attack"
2: alert = "Brute Force Attack"
3: alert = "Malware Attack"
4: alert = "Phishing Attack"
5: alert = "Normal Traffic"
```

Advantages:

- Multiple conditions can be handled clearly.
- Attack classification becomes easier.
- New attack categories can be added.
- Code becomes more readable.
- Execution can be optimized using jump tables.
- Different responses can be assigned to different attacks.

(iii) Importance of compiler optimization in real-time cybersecurity:

Compiler optimization is important because intrusion detection systems must analyze network traffic quickly.

It helps to:

- Reduce threat detection time.
- Improve packet-processing speed.
- Reduce CPU and memory usage.
- Remove unnecessary calculations.
- Improve real-time monitoring.
- Provide faster security alerts.
- Handle large amounts of network traffic.

Thus, compiler optimization helps provide fast and reliable cyberattack detection.

3. Digital Payment Gateway System

(i) Intermediate code for payment verification and fraud detection

Suppose a payment is accepted only when the amount is valid and fraud is not detected.

```
T1 = amount > 0
T2 = account_valid == TRUE
T3 = T1 AND T2

if T3 goto L1
goto L4

L1: T4 = unusual_transaction == TRUE

    if T4 goto L2
    goto L3

L2: status = "Fraud Detected"
    call rollbackTransaction
    goto L5

L3: call processPayment
    status = "Payment Successful"
    goto L5

L4: status = "Payment Rejected"

L5: ...
```

The intermediate code represents verification, fraud detection, payment processing, and rollback operations.

(ii) Backpatching in conditional transaction processing

Backpatching is used to fill the target addresses of conditional jumps after the required code locations are known.

For example:

```
if amount > 0 goto _
goto _
```

The compiler later patches the first jump to the payment verification code and the second jump to the rejection code.

Backpatching helps to:

- Generate correct conditional branches.
- Handle complex Boolean expressions.
- Connect fraud detection to the correct action.
- Implement rollback operations correctly.
- Avoid incorrect execution paths.
- Improve reliability of transaction processing.

Correct backpatching is especially important because an incorrect branch could cause a payment to be wrongly accepted, rejected, or rolled back.

(iii) Role of compiler design in secure and scalable fintech applications

Compiler design contributes to fintech systems by:

- Generating efficient intermediate code.
- Optimizing transaction processing.
- Reducing processing delays.
- Handling large numbers of transactions.
- Improving memory and CPU utilization.
- Supporting secure validation logic.
- Reducing programming errors.
- Improving scalability of payment systems.

Therefore, effective compiler design helps fintech applications become secure, fast, reliable, and scalable.

4. Smart Agriculture Monitoring System

(i) Intermediate code for irrigation control

Suppose irrigation is activated when soil moisture is low and temperature is high.

```
T1 = soil_moisture < 30
T2 = temperature > 35
T3 = T1 AND T2

if T3 goto L1
goto L2

L1: pump = ON
   call startIrrigation
   goto L3

L2: pump = OFF
```

L3: ...

This intermediate code continuously checks environmental conditions and controls the irrigation system.

(ii) Role of Boolean expressions in automated farming

Boolean expressions allow the system to make automatic decisions based on sensor values.

Example:

soil_moisture < 30 AND temperature > 35

If the expression is true, irrigation is started.

Boolean expressions are useful for:

- Checking soil moisture.
- Checking temperature.
- Detecting rainfall conditions.
- Controlling water pumps.
- Activating or stopping irrigation.
- Combining multiple sensor conditions.
- Automating farming decisions.

Thus, Boolean expressions are essential for automatic and intelligent agricultural control.

(iii) Compiler optimization in smart agriculture

Compiler optimization improves smart agriculture systems by:

- Reducing sensor-processing time.
- Providing faster irrigation response.
- Reducing unnecessary instructions.
- Saving processor memory.
- Reducing energy consumption.
- Improving embedded-controller performance.
- Supporting real-time environmental monitoring.

Therefore, optimization helps provide efficient water usage, faster control, and improved agricultural productivity.

5. Cloud-Based Video Streaming Platform

(i) Intermediate code for bandwidth allocation and quality adaptation

Suppose video quality depends on available bandwidth.

```
if bandwidth >= 10 Mbps goto L1
if bandwidth >= 5 Mbps goto L2
if bandwidth >= 2 Mbps goto L3
goto L4

L1: quality = "1080p"
   call allocateHighBandwidth
   goto L5

L2: quality = "720p"
   call allocateMediumBandwidth
   goto L5

L3: quality = "480p"
   call allocateLowBandwidth
   goto L5

L4: quality = "360p"
   call allocateMinimumBandwidth

L5: ...
```

This intermediate code allows the streaming engine to select suitable video quality based on network conditions.

(ii) Case statements for streaming quality levels

Case statements can be used to select video quality according to a quality level.

```
case quality_level

1: quality = "1080p"
2: quality = "720p"
3: quality = "480p"
4: quality = "360p"
5: quality = "240p"
```

Advantages:

- Handles multiple quality levels clearly.
- Makes the program easier to understand.
- Allows new quality levels to be added easily.
- Provides faster selection of quality.
- Can be optimized using jump tables.
- Helps manage different network conditions.

(iii) Significance of compiler optimization in cloud-based multimedia systems

Compiler optimization is important for streaming platforms because millions of users may access videos simultaneously.

Optimization helps to:

- Reduce processing time.
- Improve bandwidth management.
- Reduce buffering.
- Improve video quality.
- Reduce CPU and memory usage.
- Improve server efficiency.
- Support a large number of users.
- Improve scalability of cloud infrastructure.

SET 3

1. Autonomous Drone Delivery System

(i) Intermediate code for drone navigation and obstacle detection

Suppose the drone moves toward the destination when the path is clear and the battery is sufficient.

```
T1 = obstacle_distance < 10
T2 = battery_level > 20
T3 = T1 OR NOT T2

if T3 goto L1
goto L2

L1: call emergencyLanding
    status = "Emergency"
    goto L3

L2: call adjustRoute
    position = new_position
    status = "Navigation Continued"

L3: ...
```

This intermediate code checks obstacles and battery level before allowing normal navigation.

(ii) Boolean expressions in autonomous decision-making

Boolean expressions allow drones to make decisions automatically without human intervention.

Example:

obstacle_distance > 10 AND battery_level > 20

If both conditions are true, the drone continues its route. Otherwise, it can change its route or perform an emergency landing.

Boolean expressions help in:

- Obstacle detection.
- Battery monitoring.
- Route selection.
- Emergency landing.
- Weather-condition checking.
- Safe destination navigation.

Thus, Boolean expressions are essential for safe and autonomous drone operation.

(iii) Importance of compiler optimization in real-time delivery systems

Compiler optimization improves drone performance by:

- Reducing execution time.
- Making route adjustments faster.
- Providing quick emergency responses.
- Reducing unnecessary instructions.
- Saving memory and battery power.
- Improving processor utilization.
- Meeting real-time control requirements.

Therefore, compiler optimization helps make drone delivery faster, safer, and more reliable.

2. Railway Reservation and Scheduling System

(i) Intermediate code for reservation and scheduling

Suppose the system checks seat availability and then allocates a seat.

```
if seats_available > 0 goto L1
goto L4
```

```
L1: call allocateSeat
    seat = RET

    call updatePassengerStatus
    status = "Confirmed"

    call updateTrainSchedule
    goto L5

L4: status = "Waiting List"

L5: ...
```

For ticket cancellation:

```
if ticket_status == "Confirmed" goto L6
goto L7

L6: call cancelTicket
    call releaseSeat
    status = "Cancelled"
    goto L8

L7: status = "Cancellation Not Allowed"

L8: ...
```

(ii) Case statements for passenger ticket categories

Case statements efficiently handle different passenger categories.

```
case ticket_type

1: fare = normal_fare
2: fare = student_fare
3: fare = senior_fare
4: fare = child_fare
5: fare = special_fare
```

Advantages:

- Handles multiple categories clearly.
- Improves program readability.
- Makes fare calculation easier.
- New ticket categories can be added easily.
- Reduces complicated nested if-else statements.
- Compiler optimization can make selection faster.

(iii) Compiler optimization in large-scale transportation systems

Compiler optimization is important because railway systems process thousands of reservations simultaneously.

It helps to:

- Reduce reservation processing time.
- Speed up ticket confirmation.
- Improve cancellation processing.
- Reduce memory usage.
- Improve CPU utilization.
- Handle peak booking loads.
- Improve system scalability.
- Reduce transaction errors.

Therefore, compiler optimization provides fast, reliable, and scalable railway reservation services.

3. Intelligent Energy Management System

(i) Intermediate code for load balancing and fault detection

Suppose the system checks power load and detects faults.

```
T1 = power_load > 90
T2 = fault_detected == TRUE
T3 = T1 OR T2

if T3 goto L1
goto L2

L1: call redistributePower
   load_status = "Load Redistributed"
   goto L3

L2: load_status = "Normal"

L3: ...
```

This intermediate code allows the system to redistribute power when the load becomes too high or a fault is detected.

(ii) Significance of backpatching in smart energy management

Backpatching is used to fill the target addresses of conditional jumps after the destination labels are known.

Example:

if power_load > 90 goto _
goto _

The compiler later fills the blank destinations with the correct load-management or normal-operation addresses.

Backpatching helps to:

- Generate correct conditional branches.
- Handle Boolean expressions.
- Connect faults to recovery procedures.
- Avoid incorrect execution paths.
- Improve fault-response reliability.
- Support dynamic control decisions.

Correct backpatching is important because delayed or incorrect fault handling can affect the stability of the power system.

(iii) Compiler optimization and smart-grid performance

Compiler optimization improves smart-grid performance by:

- Reducing fault detection time.
- Speeding up power redistribution.
- Reducing unnecessary computations.
- Improving processor efficiency.
- Reducing memory requirements.
- Supporting real-time monitoring.
- Improving energy utilization.
- Increasing system reliability.

Thus, compiler optimization helps smart grids provide fast, efficient, stable, and reliable power management.

4. AI-Based Fraud Detection System

(i) Intermediate code for fraud detection and alert generation

Suppose a transaction is suspicious when the amount is high and the transaction is unusual.

T1 = transaction_amount > 100000

```
T2 = unusual_location == TRUE
T3 = failed_attempts > 3
T4 = T1 AND T2
T5 = T4 OR T3

if T5 goto L1
goto L2

L1: alert = "Fraud Suspected"
    call generateAlert
    goto L3

L2: alert = "Transaction Normal"

L3: ...
```

This intermediate code checks multiple fraud conditions and generates an alert when suspicious activity is detected.

(ii) Procedure calls and modularity

Procedure calls divide the fraud detection system into independent modules.

Example:

```
call validateTransaction
call analyzeTransaction
call calculateRisk
call generateAlert
call blockAccount
```

Benefits:

- Each procedure performs a specific function.
- Procedures can be reused.
- Testing becomes easier.
- Maintenance becomes simpler.
- Errors can be isolated.
- New fraud detection methods can be added easily.
- Different procedures can be optimized independently.

Thus, procedure calls improve the modularity, maintainability, and scalability of fraud detection software.

(iii) Compiler design in secure financial applications

Compiler design plays an important role in financial security by:

- Generating efficient and reliable code.

- Optimizing fraud detection operations.
- Reducing transaction processing time.
- Handling large transaction volumes.
- Supporting accurate conditional decisions.
- Reducing software execution errors.
- Improving resource utilization.
- Supporting scalable financial applications.

Therefore, good compiler design helps financial applications become fast, reliable, secure, and scalable.

5. Smart Warehouse Inventory Management System

(i) Intermediate code for inventory tracking and stock updates

Suppose the warehouse updates stock after an order.

```
T1 = stock_quantity - order_quantity
stock_quantity = T1

if stock_quantity < reorder_level goto L1
goto L2

L1: call generateRestockRequest
    status = "Restock Required"
    goto L3

L2: status = "Stock Available"

L3: ...
```

For product movement:

```
T2 = current_location
current_location = new_location
call updateInventoryRecord
```

This intermediate code updates stock levels and automatically generates a restocking request when inventory becomes low.

(ii) Case statements for product categories

Case statements can efficiently handle different product categories.

```
case product_category
```

1: storage_area = "Electronics"
2: storage_area = "Clothing"
3: storage_area = "Grocery"
4: storage_area = "Furniture"
5: storage_area = "Other"

Advantages:

- Handles multiple categories clearly.
- Makes warehouse logic easier to understand.
- Reduces complex if-else structures.
- Allows easy addition of new categories.
- Supports faster category selection.
- Helps assign products to appropriate storage areas.

(iii) Importance of compiler optimization in warehouse automation

Compiler optimization improves warehouse automation by:

- Speeding up inventory updates.
- Improving stock synchronization.
- Reducing order-processing delays.
- Improving automated storage control.
- Reducing memory and CPU usage.
- Improving robot response time.
- Reducing inventory errors.
- Supporting large numbers of simultaneous orders.

SET 4

1. Smart Home Automation System

(i) Intermediate code for smart device control and sensor monitoring

Suppose the lights are switched ON when it is dark, and the security alarm is activated when motion is detected while the system is armed.

T1 = light_level < 30
T2 = user_present == TRUE
T3 = T1 AND T2

if T3 goto L1
goto L2

L1: light = ON

```
    goto L3

L2: light = OFF

L3: T4 = motion_detected == TRUE
    T5 = security_armed == TRUE
    T6 = T4 AND T5

    if T6 goto L4
    goto L5

L4: call activateAlarm
    goto L6

L5: alarm = OFF

L6: ...
```

(ii) Boolean expressions in smart-home decision-making

Boolean expressions allow smart-home devices to make automatic decisions based on sensor inputs.

Example:

motion_detected == TRUE AND security_armed == TRUE

If both conditions are true, the security alarm is activated.

Boolean expressions are useful for:

- Automatic lighting control.
- Temperature control.
- Motion detection.
- Security alarm activation.
- Door and window monitoring.
- Energy-saving decisions.

Thus, Boolean expressions help smart homes make fast and automatic decisions without continuous human control.

(iii) Role of compiler optimization

Compiler optimization improves home automation by:

- Reducing device response time.
- Making security alerts faster.
- Reducing unnecessary instructions.
- Improving memory utilization.

- Reducing processor workload.
- Saving energy.
- Improving reliability of embedded controllers.

Therefore, compiler optimization makes smart-home systems faster, efficient, responsive, and reliable.

2. Online Examination Management System

(i) Intermediate code for authentication and evaluation

Suppose a student can enter the examination only when the username and password are valid.

```
T1 = username_valid == TRUE
T2 = password_valid == TRUE
T3 = T1 AND T2

if T3 goto L1
goto L2

L1: call loadExam
    call evaluateAnswers
    score = obtained_marks
    result = "Result Generated"
    goto L3

L2: result = "Authentication Failed"

L3: ...
```

This intermediate code performs authentication, exam loading, answer evaluation, and result generation.

(ii) Case statements for examination categories

Case statements can manage different examination types efficiently.

```
case exam_type

1: duration = 60
   marks = 50

2: duration = 90
   marks = 75

3: duration = 120
```

marks = 100

4: duration = 180
marks = 150

Case statements are useful because:

- Different exam categories can be handled clearly.
- Code becomes easier to understand.
- New examination categories can be added easily.
- Complex nested if-else statements can be avoided.
- Selection can be optimized by the compiler.
- Examination rules can be maintained easily.

(iii) Importance of compiler optimization

Compiler optimization is important in large-scale online examination systems because thousands of students may access the system simultaneously.

It helps to:

- Load questions faster.
- Process answers quickly.
- Generate results efficiently.
- Reduce CPU and memory usage.
- Improve server performance.
- Reduce response time.
- Support large numbers of simultaneous users.
- Improve system scalability.

Thus, optimization provides a fast, reliable, and scalable online examination system.

3. Intelligent Weather Forecasting System

(i) Intermediate code for weather monitoring and alerts

Suppose an emergency alert is generated when rainfall is high or temperature becomes extreme.

T1 = rainfall > 100
T2 = temperature > 45
T3 = temperature < 5
T4 = T2 OR T3
T5 = T1 OR T4

if T5 goto L1

```
goto L2

L1: alert = "Severe Weather"
    call sendEmergencyAlert
    goto L3

L2: alert = "Normal Weather"

L3: ...
```

The intermediate code evaluates environmental conditions and generates an alert when dangerous conditions are detected.

(ii) Role of backpatching in emergency weather alerts

Backpatching is used to fill the target addresses of conditional jumps when the final destination is not yet known.

Example:

```
if rainfall > 100 goto _
goto _
```

After the alert code is generated, the compiler fills the blank target addresses.

Backpatching helps to:

- Generate correct conditional branches.
- Handle complex Boolean expressions.
- Connect dangerous conditions to alert procedures.
- Prevent incorrect jumps.
- Improve emergency response.
- Generate reliable intermediate code.

Correct backpatching is important because incorrect branching could delay warnings during severe weather.

(iii) Compiler optimization in real-time forecasting

Compiler optimization improves forecasting applications by:

- Processing sensor data faster.
- Reducing alert generation time.
- Improving CPU utilization.
- Reducing memory requirements.
- Eliminating unnecessary calculations.
- Supporting real-time weather monitoring.
- Improving system responsiveness.
- Providing faster emergency notifications.

Therefore, optimization helps weather systems provide quick and reliable forecasting and emergency alerts.

4. Automated Library Management System

(i) Intermediate code for book issue and return

Suppose a book is issued only when it is available and the member is valid.

```
T1 = member_valid == TRUE
T2 = book_available == TRUE
T3 = T1 AND T2

if T3 goto L1
goto L2

L1: call issueBook
    book_status = "Issued"
    call updateMemberRecord
    goto L3

L2: book_status = "Issue Not Allowed"

L3: ...
```

For returning a book:

```
if book_status == "Issued" goto L4
goto L5

L4: call returnBook
    call calculateFine
    call updateBookRecord
    book_status = "Available"
    goto L6

L5: status = "Invalid Return"

L6: ...
```

(ii) Procedure calls and modularity

Procedure calls divide the library system into independent modules.

Examples:

call verifyMember
call issueBook
call returnBook
call calculateFine
call updateCatalog

Advantages:

- Each operation has a separate function.
- Procedures can be reused.
- Testing becomes easier.
- Maintenance becomes simpler.
- Errors can be identified easily.
- New library services can be added.
- The overall system becomes more organized.

(iii) Significance of compiler optimization

Compiler optimization improves educational information systems by:

- Speeding up book transactions.
- Reducing delays in fine calculation.
- Improving database update operations.
- Reducing CPU and memory usage.
- Improving response time.
- Supporting many users simultaneously.
- Improving reliability during examination periods.

Therefore, compiler optimization helps provide fast, accurate, and reliable library services.

5. Industrial Water Quality Monitoring System

(i) Intermediate code for contamination detection and alert handling

Suppose an alert is generated when pH is outside the safe range or contamination is detected.

T1 = pH < 6.5
T2 = pH > 8.5
T3 = T1 OR T2
T4 = contamination_detected == TRUE
T5 = T3 OR T4

```
if T5 goto L1
goto L2

L1: alert = "Water Contamination Detected"
    call activateWarning
    call startCorrectiveAction
    goto L3

L2: alert = "Water Quality Normal"

L3: ...
```

This intermediate code continuously checks water-quality conditions and activates corrective actions when required.

(ii) Case statements for different water quality conditions

Case statements can classify different water-quality conditions.

```
case water_quality

1: status = "Safe"
2: status = "Moderate Contamination"
3: status = "High Contamination"
4: status = "Critical Contamination"
```

They are useful because:

- Multiple conditions can be handled clearly.
- Different corrective actions can be assigned.
- Code becomes easier to read.
- New water-quality categories can be added.
- Complex if-else structures are reduced.
- The compiler can optimize category selection.

(iii) Importance of compiler optimization

Compiler optimization is very important in industrial environmental monitoring because contamination must be detected quickly.

It helps to:

- Reduce contamination detection time.
- Generate warnings faster.
- Speed up corrective actions.
- Reduce unnecessary computations.
- Improve embedded-controller performance.
- Reduce memory and CPU usage.
- Improve system reliability.

- Support continuous real-time monitoring.

SET 5

1. Smart Parking Management System

(i) Intermediate code for parking slot allocation and fee calculation

Suppose a vehicle is assigned a slot when parking is available. The parking fee depends on the vehicle type.

```
T1 = available_slots > 0
T2 = security_verified == TRUE
T3 = T1 AND T2

if T3 goto L1
goto L2

L1: call allocateSlot
    slot = RET

    if vehicle_type == "Car" goto L3
    if vehicle_type == "Bike" goto L4
    goto L5

L3: fee = hours * 50
    goto L6

L4: fee = hours * 20
    goto L6

L5: fee = hours * 100

L6: call processPayment
    status = "Parking Confirmed"
    goto L7

L2: status = "Parking Not Available"
```

L7: ...

(ii) Boolean expressions and case statements

Boolean expressions help the system check parking conditions.

Example:

available_slots > 0 AND security_verified == TRUE

The vehicle can be allocated a slot only when both conditions are true.

Case statements can classify vehicles:

```
case vehicle_type
   Car:  fee = hours * 50
   Bike: fee = hours * 20
   Bus:  fee = hours * 100
```

They help by:

- Checking parking availability.
- Verifying vehicle security.
- Selecting the correct parking slot.
- Calculating fees based on vehicle type.
- Handling multiple vehicle categories.
- Making the program easier to maintain.

(iii) Role of compiler optimization

Compiler optimization improves smart parking systems by:

- Speeding up slot allocation.
- Reducing payment-processing delays.
- Reducing unnecessary instructions.
- Improving CPU and memory usage.
- Providing faster vehicle entry and exit.
- Improving real-time response.
- Handling peak-hour traffic efficiently.

Therefore, compiler optimization makes parking systems faster, efficient, and reliable.

2. Digital Healthcare Appointment System

(i) Intermediate code for appointment scheduling and patient verification

Suppose an appointment can be booked only when the patient is verified and the doctor is available.

```
T1 = patient_valid == TRUE
T2 = doctor_available == TRUE
T3 = T1 AND T2

if T3 goto L1
goto L2

L1: call verifyPatient
    call bookAppointment
    appointment_status = "Confirmed"
    call generateReport
    goto L3

L2: appointment_status = "Booking Failed"

L3: ...
```

For cancellation:

```
if appointment_status == "Confirmed" goto L4
goto L5

L4: call cancelAppointment
    appointment_status = "Cancelled"
    goto L6

L5: appointment_status = "Invalid Cancellation"

L6: ...
```

(ii) Procedure calls and modularity

Procedure calls divide healthcare operations into separate modules.

```
call verifyPatient
call checkDoctorAvailability
call bookAppointment
call cancelAppointment
call generateReport
```

Advantages:

- Each procedure performs one task.
- Procedures can be reused.

- Testing becomes easier.
- Maintenance becomes simpler.
- Errors can be isolated.
- New healthcare functions can be added easily.
- The system becomes organized and modular.

(iii) Importance of compiler optimization

Compiler optimization improves digital healthcare systems by:

- Reducing appointment confirmation time.
- Speeding up patient verification.
- Improving report generation.
- Reducing CPU and memory usage.
- Supporting many users simultaneously.
- Improving system response time.
- Increasing reliability.

Thus, optimization helps provide fast, reliable, and efficient healthcare services.

3. Automated Toll Collection System

(i) Intermediate code for vehicle identification and toll payment

Suppose the toll depends on the vehicle type and payment is processed after vehicle verification.

```
T1 = vehicle_detected == TRUE
T2 = vehicle_valid == TRUE
T3 = T1 AND T2

if T3 goto L1
goto L5

L1: if vehicle_type == "Car" goto L2
   if vehicle_type == "Bus" goto L3
   goto L4

L2: toll = 50
   goto L6

L3: toll = 100
```

```
    goto L6

L4: toll = 150

L6: call processPayment
    payment_status = "Successful"
    call openGate
    goto L7

L5: payment_status = "Vehicle Not Verified"

L7: ...
```

(ii) Significance of backpatching

Backpatching is used to fill the target addresses of conditional jumps after the final locations of the target statements are known.

Example:

```
if vehicle_valid == TRUE goto _
goto _
```

The compiler later fills the blank targets with the correct payment or rejection code.

Backpatching helps to:

- Generate correct conditional jumps.
- Handle Boolean expressions.
- Connect vehicle validation to payment processing.
- Prevent incorrect execution paths.
- Improve transaction reliability.
- Reduce errors in intermediate code.

Correct backpatching is important because incorrect branching could delay toll payment and vehicle movement.

(iii) Compiler optimization in transportation applications

Compiler optimization improves large-scale transportation systems by:

- Speeding up vehicle identification.
- Reducing payment processing time.
- Improving toll-gate response.
- Reducing CPU and memory requirements.
- Handling large traffic volumes.
- Reducing traffic congestion.
- Improving transaction reliability.

Therefore, optimization supports fast and efficient automated transportation systems.

4. Intelligent Fire Detection and Alarm System

(i) Intermediate code for fire detection and emergency alerts

Suppose an alarm is activated when smoke is detected or temperature exceeds the safe limit.

```
T1 = smoke_level > 70
T2 = temperature > 60
T3 = T1 OR T2

if T3 goto L1
goto L2

L1: alarm = ON
    call activateAlarm
    call emergencyProtocol
    goto L3

L2: alarm = OFF

L3: ...
```

This intermediate code continuously checks sensor conditions and activates emergency procedures when required.

(ii) Case statements for emergency conditions

Case statements can classify different emergency conditions.

```
case emergency_level

1: status = "Normal"
2: status = "Warning"
3: status = "Fire Detected"
4: status = "Critical Fire Emergency"
```

Each case can perform a different action:

- Normal → Continue monitoring.
- Warning → Send warning notification.
- Fire Detected → Activate alarm.
- Critical → Activate alarm and emergency evacuation.

Advantages:

- Handles multiple conditions clearly.
- Makes emergency logic easier to understand.
- Avoids complex nested if-else statements.
- Allows new emergency conditions to be added.
- Can be optimized by the compiler.
- Provides quick selection of emergency actions.

(iii) Compiler optimization in industrial safety systems

Compiler optimization is extremely important in safety systems because every second can be critical.

It helps to:

- Detect fires faster.
- Reduce alarm response time.
- Reduce unnecessary calculations.
- Improve sensor-processing speed.
- Improve CPU utilization.
- Reduce memory usage.
- Support reliable emergency procedures.

Therefore, optimized compiler code improves industrial safety, emergency response, and reliability.

\

---

5. AI-Based Customer Support Chatbot

(i) Intermediate code for chatbot query processing

Suppose the chatbot identifies the type of customer query and generates an appropriate response.

```
T1 = query_contains("order") == TRUE
T2 = query_contains("refund") == TRUE
T3 = query_contains("technical") == TRUE

if T1 goto L1
if T2 goto L2
if T3 goto L3
goto L4

L1: call processOrderQuery
    response = "Order information provided"
    goto L5
```

```
L2: call processRefundQuery
   response = "Refund information provided"
   goto L5

L3: call processTechnicalQuery
   response = "Technical support provided"
   goto L5

L4: call processGeneralQuery
   response = "General assistance provided"

L5: call sendResponse
```

This intermediate representation allows different types of queries to be processed using separate procedures.

(ii) Procedure calls and modularity

Procedure calls divide the chatbot into independent modules.

```
call analyzeQuery
call processOrderQuery
call processRefundQuery
call processTechnicalQuery
call processGeneralQuery
call sendResponse
```

Advantages:

- Each module performs a specific task.
- Modules can be reused.
- Testing becomes easier.
- New query types can be added easily.
- Individual modules can be updated independently.
- Errors can be isolated.
- The chatbot becomes easier to maintain and scale.

(iii) Importance of compiler optimization

Compiler optimization is important for intelligent customer-support systems because many users may interact with the chatbot simultaneously.

Optimization helps to:

- Reduce response time.
- Process queries faster.
- Reduce unnecessary computations.
- Improve CPU and memory utilization.
- Handle a large number of users.

- Improve server efficiency.
- Reduce response delays.
- Improve overall user experience.

RUBRICS FOR CALCULATION

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate;	Minor calculation errors	Frequent errors; incorrect	No valid calculations

		correct final answer		final answer	
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation